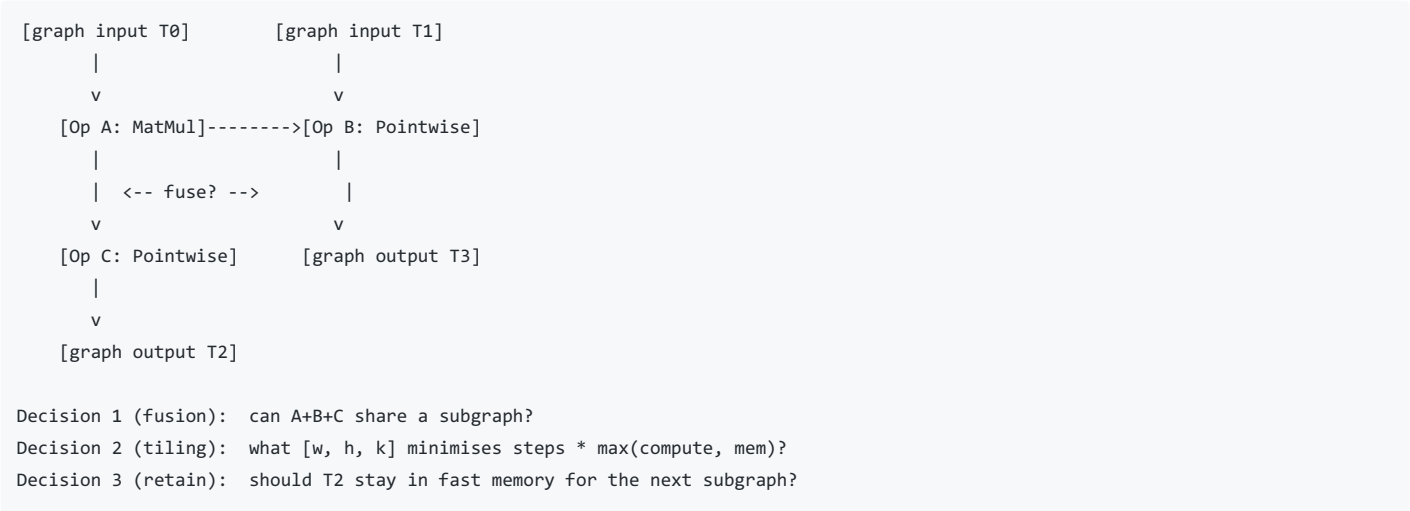


MLSys 2026 Track A - Scheduler Inspired by Bitbake

The core challenge in this track is scheduling a directed acyclic graph (DAG) of tensor operations onto hardware that has a small, fast on-chip memory and a large, slow off-chip DRAM. Every time a tensor must leave fast memory and later be reloaded, it costs bandwidth; the scoring function penalises exactly this traffic. Solving the problem well requires three interleaved decisions at once: which operations to fuse into a single subgraph (so that intermediate tensors never touch DRAM at all), what tile granularity to execute at (balancing compute overhead against memory footprint), and which output tensors to keep warm in fast memory across subgraph boundaries (avoiding one evict-plus-reload cycle per handoff).

Figure 1: The three-tier scheduling decision for a small DAG.



While designing this scheduler I recognised that BitBake - the build system used by the Yocto Project for embedded Linux - faces structurally the same problem. BitBake must decide which recipe tasks to group together (do_configure and do_compile share a build directory, so grouping them avoids re-staging source), which intermediate artefacts are private to a recipe and can therefore be discarded after the group finishes, and whether a previously built artefact already lives in the sstate cache (fast, local) or must be fetched from a remote mirror (slow, expensive). The mapping is direct: DEPENDS edges in a BitBake recipe correspond to tensor data edges; task grouping corresponds to op fusion; an sstate cache hit - leaving an artefact in /tmp because the next recipe will need it immediately - corresponds to tensors_to_retain; and a build artefact that is too large for /tmp scratch space corresponds to a working set that exceeds fast_memory_capacity. This analogy was the guiding design principle throughout.

BitBake: Philosophy and Inspiration

BitBake is a task execution engine at the heart of the Yocto Project. Its central job is to take a large graph of interdependent recipes, each with multiple tasks (fetch, unpack, configure, compile, install, package), and execute them in an order that satisfies all DEPENDS and RDEPENDS edges while making the best possible use of shared state (sstate) caches and disk scratch space. Several of its design principles map directly onto the tensor scheduling problem addressed in this contest.

Figure 2: Concept mapping between BitBake and this scheduler.

BitBake concept	Scheduler concept
DEPENDS / RDEPENDS edge	tensor data edge in the op DAG
recipe task group	fused subgraph
intermediate artefact (private)	ephemeral tensor (never hits DRAM)
sstate cache (fast, local)	tensors_to_retain in fast memory
remote mirror (slow)	slow DRAM (off-chip bandwidth)
/tmp scratch space limit	fast_memory_capacity
greedy task-group heuristic	greedy single-pass fusion loop
two-phase sstate eviction	two-pass retention post-step

Task grouping as fusion. BitBake recipes often declare that two tasks must run in the same working directory because one produces an artefact the next consumes immediately (e.g. do_configure writes Makefiles that do_compile reads). Grouping them avoids re-staging files between tasks. The scheduler here applies exactly the same logic: if op B's input tensor is the output of op A, and no other op outside the group needs that tensor, the two ops are fused into a single subgraph so the intermediate tensor never has to be written to DRAM.

Shared state cache as tensor retention. BitBake's sstate cache lets a task skip re-execution if a valid cached result already exists in a fast local storage location. When a recipe finishes, BitBake decides whether to evict the result to a remote (slow) mirror or leave it in the local cache for the next recipe that will need it. The

`tensors_to_retain` mechanism here is the same decision: a tensor that will be consumed immediately by the next subgraph is marked as retained, meaning it stays in fast on-chip memory and its store cost (bandwidth to slow DRAM) is skipped entirely.

Scratch space capacity as fast_memory_capacity. Every build machine has a finite /tmp or build partition. BitBake must ensure that the aggregate footprint of all live intermediate artefacts at any moment does not overflow that space. This is the `working_set` guard in the scheduler: before accepting a fusion, the scheduler computes the total tile footprint of all boundary tensors and rejects the merge if it would exceed `fast_memory_capacity`. The analogy is precise because both systems are managing a finite fast-storage budget against an unbounded slow-storage backend.

DEPENDS edges as tensor data edges. In a BitBake recipe, a DEPENDS declaration states that one recipe must be fully built before another can begin. This creates the same kind of directed acyclic dependency graph that tensor operations form through their input/output tensors. Both graphs require topological ordering before any scheduling decisions can be made. The dependency graph structure of a real Yocto build – where recipe nodes fan out and converge exactly as tensor ops do here – is illustrated in [6], slide 27.

The BitBake mental model was not just an analogy for documentation purposes; it actively shaped implementation choices. The decision to use a greedy single-pass fusion (rather than a global search) mirrors BitBake's own greedy task grouping: both systems favour a fast, safe heuristic over an expensive optimal search because the graphs encountered in practice are predominantly linear chains where greedy is already optimal. The two-pass retention post-step mirrors BitBake's two-phase sstate handling: first identify which artefacts will be needed soon, then adjust the eviction policy accordingly.

Algorithm

Topological ordering (`baseline.c` - `topo_sort`)

All scheduling decisions must respect data dependencies: an op cannot run until all of its input tensors have been produced. Kahn's algorithm is used to compute a valid linear ordering. It initialises a queue with all ops whose inputs are graph inputs (in-degree zero), then repeatedly dequeues an op, appends it to the order, and decrements the in-degree of every op that consumes one of its outputs. If the final count of ordered ops is less than the total op count, a cycle exists and the scheduler aborts. The result is a deterministic, linear order that serves as the foundation for every subsequent pass.

Fusion (`fusion.c` - `schedule_fusion`)

Figure 3a: Fusion - before and after.

```
BEFORE (baseline, one op per subgraph):
[T0]->[A]->[T1]->[B]->[T2]->[C]->[T3]
 ^DRAM ^DRAM ^DRAM ^DRAM ^DRAM ^DRAM

AFTER (greedy fusion, three guards pass):
[T0]->[ A + B + C ]->[T3]
 ^DRAM T1,T2 stay ^DRAM
      in fast mem
```

Figure 3b: Tensor retention between adjacent subgraphs.

```
Subgraph 0      | Subgraph 1
[Op A] -> [T_hot] | [T_hot] -> [Op B]
      \   |   /
      +-retained-+ (no DRAM round-trip)
      fast memory

Without retention: T_hot written to DRAM after subgraph 0,
                  then reloaded before subgraph 1.

With retention:   T_hot stays in fast memory; bandwidth cost skipped.
```

The scheduler walks the topological order and maintains an open subgraph. For each successive op it checks three guards before merging. First, `has_direct_edge` confirms that at least one tensor flows from the current subgraph into the candidate op - there must be an actual data dependency, not just a scheduling adjacency. Second, `intermediates_are_safe` verifies that every tensor crossing the A-to-B boundary has all of its consumers inside the merged group; if any consumer is external, the tensor must be materialised in DRAM and fusion is not beneficial. Third, `working_set` computes the total tile footprint of boundary tensors (inputs from outside the group plus outputs going outside) and checks that it does not exceed `fast_memory_capacity`. If all three pass the op is absorbed; otherwise the current subgraph is finalised and a new one is opened. Greedy is the right choice here: it is $O(n)$ in the number of ops, it handles the common linear-chain pattern optimally, and the three guards make it safe to apply without a global search.

Granularity selection (`fusion.c` - `pick_gran`)

Each subgraph runs at a three-dimensional tile size $[w, h, k]$. The spatial dimensions w and h are fixed at the native granularity of the hardware (going smaller incurs a utilisation penalty that scales as $\text{native_w} \times \text{native_h} / w \times h$). The K dimension, which governs the inner loop of MatMul, is selected as follows. For a single-op subgraph, `full_K` is used unconditionally: memory overflow is unavoidable for an op that is too large to fit on its own, and using `full_K` minimises the number of execution steps. For a multi-op fusion group, a binary-halving search is run: starting from `full_K`, the search tries `full_K`, `full_K/2`, `full_K/4`, and so on, calling `working_set(ops, nops, p, info, g)` at each candidate to check whether the boundary tile footprint fits in `fast_memory_capacity`. The largest k that fits is chosen. This is what allows chained MatMul sequences to fuse: a smaller k reduces the tile footprint of the K -dimension boundary, making room for both the LHS and RHS slices to coexist in fast memory during execution.

Tensor retention (fusion.c - post-pass)

After the subgraph partition is fixed, a two-pass post-step populates `tensors_to_retain`. In the first pass, for every adjacent pair of subgraphs (`sg[i]`, `sg[i+1]`), any tensor produced by `sg[i]` and consumed by `sg[i+1]` is added to `sg[i].tensors_to_retain`. In the second pass all subgraphs are re-evaluated, passing the previous subgraph's retain list to `evaluate_subgraph`. The evaluator uses this list in two places: it skips the load cost for retained inputs (they are already in fast memory) and it skips the store cost for retained outputs (they do not need to be evicted). This directly reduces the `mem_bytes` term in the roofline formula, lowering latency wherever a tensor would otherwise have made a round-trip to DRAM between two adjacent subgraphs.

Results

Five benchmark problems from the MLSys 2026 Track A suite were used to measure the scheduler. The table below summarises subgraph count and total latency after the full pipeline (fusion + split-K + retention).

Benchmark	Ops	Subgraphs	Total latency	Key observation
bench-1	5	5 → 1	419,430	All 5 ops fused at k=64; single subgraph
bench-5	19	19 (same)	961,195	No safe fusions; MatMuls too large to tile together
bench-9	32	32 → 8	6,223,094	4-op groups at k=512; ~3.8x reduction vs baseline
bench-13	63	49 (same)	22,156,083	Large MatMuls exceed fast mem solo; pointwise fused
bench-17	103	89 (same)	5,009,265	Mixed graph; retention saves reload on boundaries

The biggest gain comes from split-K fusion on benchmark-9, where grouping four ops at a halved K dimension reduces total latency by roughly 3.8x compared to the one-op-per-subgraph baseline. Benchmark-1 collapses all five ops into a single subgraph. Benchmarks-5, -13, and -17 are dominated by large MatMuls whose tile footprint exceeds `fast_memory_capacity` even at reduced K, leaving fusion blocked; retention provides a smaller but measurable saving on the subgraph boundaries.

Future Work

Three directions would likely improve the score further.

Traversal order optimisation. Currently all subgraphs use the default raster (row-major) traversal order. A snake or zig-zag traversal of the spatial tile grid keeps the right-hand side tile of a MatMul warm between consecutive rows, halving the RHS reload cost. The solution format already accepts a `traversal_order` array per subgraph; the scheduler just needs to emit it.

Multi-hop retention. The current post-pass only retains tensors between immediately adjacent subgraphs. A tensor needed two subgraphs later (a skip-connection pattern) could also benefit from retention, provided the fast memory budget allows it to coexist with the intervening subgraph's working set. A feasibility check against `fast_memory_capacity` would gate this safely.

Recomputation versus spilling. For diamond-shaped graphs - where one op fans out to two independent chains that later converge - the shared fanout tensor must either be spilled to DRAM (one write, one read per consumer) or recomputed from scratch in each branch. When the op is cheap and the tensor is large, recomputation wins; when the op is expensive, spilling wins. Adding a cost comparison at split nodes would cover this case, analogous to Flash-Attention's decision to recompute attention weights rather than materialise them.

References

[1] BitBake User Manual, Yocto Project. <https://docs.yoctoproject.org/bitbake/>

[2] Dave Gamble, *cJSON - Ultralightweight JSON parser in ANSI C*. <https://github.com/davegamble/cjson>

[3] Norman P. Jouppi et al., "In-Datcenter Performance Analysis of a Tensor Processing Unit", ISCA 2017. (Roofline model for matrix accelerators.)

[4] Tri Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", NeurIPS 2022. (Recomputation vs. spilling trade-off.)

[5] Samuel Williams, Andrew Waterman, David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", CACM 2009.

[6] Mamta Shukla, "Yocto vs Buildroot: A Case Study of SPEC7", CERN EP-ESE seminar, 2020. (Slide 27 shows the BitBake recipe dependency graph whose structure directly inspired the DAG scheduling approach used in this scheduler.)
https://indico.cern.ch/event/921378/contributions/3919923/attachments/2066928/3468928/Yocto_vs_Buildroot_A_case_study_of_SPEC7.pdf